

AI DEBUGGER PRO

# AI Debugger Pro: Repair Verification, Evidence, and Trust



## **ABSTRACT**

AI-assisted repair systems can generate convincing explanations and syntactically valid patches while leaving the original defect unresolved, introducing regressions, weakening security, or changing intended behavior. This paper defines a verification and trust framework for AI Debugger Pro, a human-governed desktop debugging prototype. The current implementation correctly separates diagnosis from mutation: it parses a structured model response, previews a full-source replacement as a diff, requires explicit approval, reruns approved code, and records the result as a linked history event. That workflow establishes useful evidence, but its present `verified_fix` state is narrower than ordinary readers may assume. A zero exit status proves only that one execution completed successfully under a particular backend and input. It does not establish semantic correctness, regression freedom, security, generality, or alignment with user intent. The paper introduces an evidence ladder with six levels: proposal validity, build validity, failure non-recurrence, behavioral validation, regression validation, and policy/provenance validation. It then specifies evidence objects, trust labels, confidence separation, stale-approval prevention, independent-oracle requirements, multi-language considerations, and acceptance criteria for a stronger verification engine. The central claim is that trustworthy AI-assisted repair does not arise from model confidence or human approval alone. It emerges from a chain of independently inspectable evidence whose scope is visible, whose provenance is bound to immutable inputs, and whose limitations survive into the interface and history record.

**Keywords:** automated program repair, software testing, human-in-the-loop AI, verification, provenance, regression testing, calibrated trust

# 1. The verification problem

AI Debugger Pro follows a disciplined sequence: deterministic execution produces a failure; an optional model proposes a diagnosis and replacement; the user reviews a diff; approval triggers another execution; and history records the outcome. This is substantially safer than silent autonomous mutation. Yet the word “verified” carries more weight than the current evidence supports.

In `interface/gui.py`, an approved repair is recorded as `verified_fix` when `_run` returns success. For Python, C, C++, and Java, success ordinarily corresponds to a zero process return code after syntax checking or compilation where applicable. The rerun may use the same example that exposed the failure. No repository test suite is automatically discovered or executed. The suggested regression test is displayed and can be saved, but it is not independently generated, required, or automatically run. Security policy analysis and behavioral equivalence checking are not present.

The implementation is not deceptive; the README describes an “automatic rerun” and a “verified or failed repair.” The research challenge is to make the evidence boundary explicit. A tool earns trust when it says exactly what passed, under which inputs and environment, and what remains unknown.

# 2. Scope and method

This paper analyzes verification after an AI-generated repair proposal. It covers proposal parsing, approval, rerun behavior, regression-test suggestions, execution backends, history, and user-facing status. It does not claim formal proof, benchmark results, or production assurance. Recommendations are separated from implemented behavior.

The method combines repository inspection with established ideas from automated program repair and testing: a patch must satisfy an oracle; passing tests may be incomplete; overfitting patches can satisfy observed cases without correcting the underlying defect; mutation testing can evaluate test sensitivity; reproducible builds and provenance strengthen the interpretability of evidence; and human factors affect whether evidence is understood or merely obeyed.

# 3. Current repair protocol

| Stage           | Implemented behavior                                                                | Evidence produced    | Important limitation                                         |
|-----------------|-------------------------------------------------------------------------------------|----------------------|--------------------------------------------------------------|
| Failure capture | Execute code and capture output/status; Python may capture bounded traceback locals | Concrete failing run | One input and environment may not represent the defect class |

| Stage     | Implemented behavior                                                                           | Evidence produced        | Important limitation                                        |
|-----------|------------------------------------------------------------------------------------------------|--------------------------|-------------------------------------------------------------|
| Diagnosis | Request JSON fields for summary, root cause, replacement source, explanation, test, confidence | Structured proposal      | Schema validity is not semantic validity                    |
| Review    | Show diagnosis and unified diff; enable approve/reject                                         | Visible candidate change | Full-file replacement can hide broad edits                  |
| Approval  | User explicitly selects apply-and-verify                                                       | Authorization event      | Approval can become stale if inputs change                  |
| Rerun     | Replace editor text and execute candidate                                                      | Exit status and output   | Success does not prove behavioral correctness               |
| History   | Store run and repair events with parent identifiers and code diff                              | Local audit trail        | Lacks environment, model, hash, test, and policy provenance |

The architecture has a strong trust primitive: model output has no direct mutation authority. The human approval gate is real. The missing layer is a precise contract for what the post-approval rerun can certify.

## 4. Trust is multidimensional

Trust should not be represented by a single score. At least five dimensions differ:

- **Model confidence:** the model's self-reported certainty about its diagnosis.
- **User confidence:** the reviewer's subjective belief after reading the explanation and diff.
- **Execution evidence:** what actually occurred under a specified backend, command, input, and resource policy.
- **Test evidence:** which independent assertions passed and how sensitive they are to defects.
- **Policy evidence:** whether deterministic checks detected prohibited or high-risk changes.

These values must not be averaged into a decorative percentage. Model confidence can be high when evidence is weak; a test suite can be strong even when the explanation is poor; and a patch can remove the observed crash while violating policy. The interface should preserve disagreement between dimensions because disagreement is information.

## 5. Evidence ladder

| Level | Label                           | Minimum evidence                                                      | What it permits the system to say                             |
|-------|---------------------------------|-----------------------------------------------------------------------|---------------------------------------------------------------|
| 0     | Proposed                        | Parsed candidate bound to source and failure hashes                   | "A repair was proposed."                                      |
| 1     | Build-valid                     | Syntax check or compilation succeeds                                  | "The candidate builds."                                       |
| 2     | Failure not reproduced          | Original failing case completes without the same failure signature    | "The observed failure did not recur."                         |
| 3     | Behavior-checked                | Explicit expected output, assertion, or invariant passes              | "The specified behavior passed."                              |
| 4     | Regression-checked              | Repository-owned or independently derived test set passes             | "The selected regression suite passed."                       |
| 5     | Policy-checked and reproducible | Security rules pass; environment and evidence provenance are complete | "The recorded checks passed under this reproducible profile." |

No level proves universal correctness. Each label is a scoped statement about evidence. Level 2 is the closest description of the current positive path when the original failing example is rerun. Calling it simply “verified” compresses away too much uncertainty.

## 5.1 Failure signatures

Verification should preserve a normalized signature of the original failure: language, exception or diagnostic class, relevant frame or compiler location, normalized message, return code, and input identifier. The repaired run can then establish whether the same signature disappeared. Merely observing return code zero is weaker because a patch can suppress the failing path, remove functionality, swallow exceptions, or stop calling the affected code.

## 5.2 Behavioral oracles

A behavioral check requires an oracle: an expected value, property, invariant, snapshot, differential result, or human-confirmed outcome. Good oracles are independent of the patch. A regression test supplied by the same model that produced the repair is useful as a hypothesis but not fully independent evidence; both artifacts can share the same misunderstanding.

## 5.3 Regression evidence

Repository-owned tests carry prior intent. Newly generated tests expand coverage but need review. The engine should report discovered, selected, skipped, failed, and timed-out tests separately. A single aggregate green light hides whether zero tests ran—a classic little gremlin wearing a success badge.

# 6. Evidence object and provenance

Every verification attempt should emit a structured record.

| Field group   | Required contents                                                                                      |
|---------------|--------------------------------------------------------------------------------------------------------|
| Inputs        | Original-source hash, candidate hash, failure signature, selected file paths, user-supplied input hash |
| Proposal      | Provider, exact model identifier, request/response timestamps, schema version, prompt-policy version   |
| Authorization | Reviewer action, proposal hash, approval timestamp, displayed-diff hash                                |
| Environment   | Backend, container image digest, interpreter/compiler versions, operating profile, resource limits     |
| Checks        | Check identifiers, commands, expected outcomes, actual outcomes, duration, stdout/stderr digests       |
| Outcome       | Highest evidence level reached, failed/skipped checks, residual limitations                            |
| Lineage       | Parent run, proposal, approval, and verification record identifiers                                    |

Hashes do not make evidence true, but they make substitutions detectable. An approval must be bound to the exact candidate and exact displayed diff. If source, language, failure, execution profile, or candidate changes, approval becomes invalid and the UI must return to “proposal changed.”

## 7. Independent verification strategy

### 7.1 Separate generation from judgment

The repair model should not be the sole judge of its own work. Verification should prefer deterministic execution and repository-owned tests. If another model generates tests or reviews the patch, its independence is limited unless it receives different instructions, context, or model lineage. The system must label that evidence honestly as model-assisted review, not independent proof.

### 7.2 Fresh execution environment

The candidate should run in a fresh restricted environment. Reusing mutated state can create false passes through cached files, environment changes, generated artifacts, or modified dependencies. The original and repaired program should receive equivalent inputs and resource policies unless the user intentionally changes the experiment.

### 7.3 Differential behavior

For cases that did not fail originally, the system can compare outputs between original and repaired versions. Unexpected differences become review targets. Differential checks are especially useful when a patch is broader than the failing function. They cannot determine which behavior is correct, but they expose semantic drift.

### 7.4 Metamorphic and property checks

When exact expected outputs are unavailable, properties can still constrain behavior: sorting output remains ordered and preserves elements; serialization round-trips; totals remain conserved; identifiers remain unique; equivalent inputs produce equivalent outputs. These checks generalize beyond a single example and reduce patch overfitting.

### 7.5 Mutation testing

Mutation testing can estimate whether selected tests notice nearby defects. If trivial mutations survive, a green suite provides weak evidence. This should be an optional advanced check because it increases runtime, but its result is more informative than raw test count.

## 8. Human approval as authorization

Approval is a security and governance event, not a correctness oracle. The reviewer needs a bounded diff, explanation tied to changed lines, stated assumptions, evidence plan, and warnings for risky APIs or broad rewrites. The action should remain disabled until the proposal is fully rendered and its source binding is current.

The UI should separate **Apply** from **Verify** conceptually even when they occur in one guided flow. Applying authorizes mutation. Verifying executes checks against the applied or staged candidate. A user may approve experimentation without asserting correctness, and may reject a behaviorally successful patch because it violates intent or maintainability standards.

Recommended states are: proposal ready, changed since review, approved for verification, build failed, observed failure persists, observed failure absent, behavior checked, regressions passed, policy warning, and verification incomplete. Red and green alone are insufficient; every state needs a textual scope statement.

## 9. Multi-language verification

| Language | Build evidence                             | Runtime evidence                                   | Recommended test adapters                  |
|----------|--------------------------------------------|----------------------------------------------------|--------------------------------------------|
| Python   | AST parse and interpreter start            | Exception/exit/output under selected interpreter   | pytest, unittest, doctest, property checks |
| C        | Compiler success with captured diagnostics | Exit/signal/output; sanitizer status where enabled | CTest, custom harnesses, sanitizers        |

| Language | Build evidence          | Runtime evidence                             | Recommended test adapters                     |
|----------|-------------------------|----------------------------------------------|-----------------------------------------------|
| C++      | Compiler/linker success | Exit/signal/output; sanitizer status         | CTest, Catch2/GoogleTest adapters, sanitizers |
| Java     | javac success           | JVM exit/exception/output under selected JDK | JUnit, Maven/Gradle test adapters             |

Cross-language labels must remain comparable without erasing language-specific meaning. A compiler pass is not equivalent to a test pass. Toolchain versions and flags are part of the evidence, especially because undefined behavior, optimization, dependencies, and runtime versions can change results.

## 10. Failure modes of naive verification

| Failure mode             | Example                                                   | Detection strategy                                           |
|--------------------------|-----------------------------------------------------------|--------------------------------------------------------------|
| Exception suppression    | Replace failing logic with <code>try/except</code> : pass | Require behavioral oracle and diff policy                    |
| Constant-output patch    | Return the expected demo value for every input            | Hidden/held-out and property-based cases                     |
| Feature deletion         | Remove the call that triggered the crash                  | Coverage and intent-sensitive assertions                     |
| Test weakening           | Modify assertions to accept wrong behavior                | Protect repository-owned tests; review test diffs separately |
| Environment dependence   | Pass only because cached state or host dependency exists  | Fresh pinned environment and clean workspace                 |
| Partial-language success | Compile succeeds but runtime behavior fails               | Separate build and runtime stages                            |
| Security regression      | Introduce shell, network, or secret access                | Deterministic policy checks and threat-model rules           |
| Stale approval           | Apply a candidate after source changed                    | Hash-bound authorization and invalidation                    |

## 11. Proposed verification pipeline

- 1 Freeze the repair candidate, original source, failure signature, input set, and execution profile.
- 2 Compute hashes and invalidate any approval bound to different values.
- 3 Render a minimal diff with risky-change annotations.
- 4 Record explicit authorization for this candidate only.
- 5 Create a fresh restricted workspace from immutable inputs.
- 6 Run syntax/compile checks and preserve full structured results within output bounds.
- 7 Reproduce the original case against the candidate and compare failure signatures.
- 8 Execute user-specified oracles and repository-owned tests.
- 9 Optionally execute reviewed generated tests, property checks, differential cases, mutation tests, and security policy checks.
- 10 Emit the evidence object and assign only the highest satisfied scoped label.
- 11 Preserve failures, skips, uncertainty, and environment identity in history.
- 12 Let the user accept, revise, or revert based on evidence—not on model confidence alone.

## 12. Metrics for empirical evaluation

The verification engine should be assessed separately from repair generation. Useful metrics include true repair acceptance, plausible-but-incorrect patch rejection, failure-signature resolution, regression detection, security-regression detection, test discovery accuracy, zero-test false-green rate, stale-approval prevention, reproducibility across repeated clean runs, evidence-record completeness, reviewer decision accuracy, review time, and calibration between displayed labels and actual evidence.

The existing empirical evaluation protocol can incorporate controlled ablations: rerun only; rerun plus model-generated test; rerun plus repository tests; full evidence ladder; and full ladder with policy analysis. This permits measurement of which controls improve correctness and which merely improve confidence.

## 13. Acceptance criteria

A strengthened release should not use an unqualified “verified repair” label unless:

- the proposal is bound to immutable source, failure, candidate, diff, and environment identifiers;
- any intervening edit invalidates approval;
- syntax/compilation and runtime outcomes are represented separately;
- the original failing case is identified and its post-repair status recorded;
- zero discovered tests cannot produce a “tests passed” state;
- model-generated tests are labeled by provenance and separated from repository-owned tests;
- skipped, timed-out, and unavailable checks remain visible;
- verification occurs in a fresh execution environment;
- the record states the highest satisfied evidence level and residual limitations;
- history preserves lineage without silently rewriting prior evidence.

## 14. Implementation roadmap

### Phase V1 - truthful state semantics

- Rename the current positive state to “rerun passed” or “observed failure not reproduced.”
- Add source, candidate, failure, and diff hashes.
- Invalidate proposals on every editor or language change.
- Store backend, duration, return code, and toolchain identity.

### Phase V2 - test integration

- Discover repository test frameworks conservatively and require user confirmation of commands.
- Add explicit expected-output and invariant checks for single-file examples.
- Separate repository-owned, user-authored, and model-generated tests.
- Report counts for collected, passed, failed, skipped, and not run.

### Phase V3 - policy and reproducibility

- Pin execution images and capture digests.
- Add risky-change rules derived from the security threat model.

- Run original and candidate versions in clean equivalent environments.
- Export a machine-readable evidence bundle alongside the human report.

## Phase V4 - advanced assurance

- Add differential, property-based, mutation, and flaky-test analysis.
- Measure reviewer calibration and automation bias.
- Support signed or append-only verification provenance.

## 15. Limitations

The proposed framework increases evidence quality but cannot prove general program correctness for arbitrary software. Test suites remain incomplete; specifications may be wrong; environments can differ; nondeterminism can obscure causality; and policy checks can miss novel risks. Formal methods may strengthen narrow components but are not a universal substitute for requirements and review.

This paper is a repository-grounded assurance design, not a report of implemented V1-V4 features. The current prototype provides proposal parsing, visible diffs, explicit approval, rerun evidence, suggested-test handling, and linked local history. All stronger labels and evidence structures described here are recommendations until implemented and tested.

## 16. Conclusion

AI Debugger Pro begins from the right governance principle: proposal is not action. Its next leap is to make success equally disciplined: execution is not correctness, a generated test is not an independent oracle, confidence is not evidence, and approval is not proof.

A trustworthy repair workflow should behave like a careful scientific instrument. It freezes inputs, records conditions, distinguishes observation from inference, exposes uncertainty, and allows another person to reproduce the result. Under that model, “verified” stops being a green glow and becomes a compact statement backed by inspectable facts. The system need not promise perfection. It must promise that its evidence has a name, a scope, a lineage, and no fake moustache.

## Appendix A. Repository evidence map

| Concern                                      | Repository evidence                                            |
|----------------------------------------------|----------------------------------------------------------------|
| Structured diagnosis fields and confidence   | core/diagnostics.py;<br>core/ai_suggester.py::_parse_diagnosis |
| Instruction not to claim verification        | core/ai_suggester.py::ai_diagnose prompt                       |
| Diff preview and pending proposal            | interface/gui.py::_receive_diagnosis                           |
| Explicit approval/rejection                  | interface/gui.py::apply_and_verify; reject_patch               |
| Positive repair event based on rerun success | interface/gui.py::apply_and_verify                             |
| Suggested regression test is displayed/saved | interface/gui.py::save_regression_test                         |
| Event lineage and diffs                      | core/history.py::ExecutionHistory.add                          |
| Language-specific execution                  | core/executor.py; core/sandbox.py                              |

## Appendix B. References

- 1 M. Monperrus. “Automatic Software Repair: A Bibliography.” *ACM Computing Surveys*, 2018. <https://doi.org/10.1145/3105906>

- 2 E. K. Smith et al. "Is the Cure Worse Than the Disease? Overfitting in Automated Program Repair." *ESEC/FSE*, 2015.  
<https://doi.org/10.1145/2786805.2786825>
- 3 G. Fraser and A. Zeller. "Mutation-Driven Generation of Unit Tests and Oracles." *IEEE Transactions on Software Engineering*, 2012.  
<https://doi.org/10.1109/TSE.2011.93>
- 4 P. Ammann and J. Offutt. *Introduction to Software Testing*, 2nd ed. Cambridge University Press, 2016.
- 5 SLSA. *Supply-chain Levels for Software Artifacts, Provenance*. <https://slsa.dev/spec/>
- 6 National Institute of Standards and Technology. *Artificial Intelligence Risk Management Framework*. NIST AI 100-1, 2023.  
<https://doi.org/10.6028/NIST.AI.100-1>
- 7 T. R. Bentley. *AI Debugger Pro: Current Architecture and Human-Governed Repair Protocol*. September 2026.
- 8 T. R. Bentley. *AI Debugger Pro: Security Threat Model for Human-Governed AI-Assisted Repair*. September 2026.

## Appendix C. Repair-review checklist

Before authorizing a candidate, the reviewer should be able to answer the following questions from visible evidence:

- 1 Is the proposal still bound to the source and failure currently displayed?
- 2 Does the diff modify only files and regions relevant to the diagnosed defect?
- 3 Are deletions, exception suppression, constant outputs, disabled checks, or weakened assertions explained?
- 4 Did the candidate introduce network, process, filesystem, dynamic-evaluation, credential, or permission behavior?
- 5 Is the original failure case preserved as a reproducible input rather than silently removed?
- 6 Which checks are repository-owned, user-authored, model-generated, skipped, unavailable, or timed out?
- 7 Did any test run, or is a zero-test collection being mistaken for success?
- 8 Were the original and candidate executed under equivalent fresh environments and resource policies?
- 9 Do output differences outside the failing case require human interpretation?
- 10 Does the final label state what passed without implying universal correctness?

| Display label      | Reviewer interpretation                                                          |
|--------------------|----------------------------------------------------------------------------------|
| Proposal ready     | Untrusted candidate available; no execution evidence yet                         |
| Build passed       | Syntax or compilation succeeded; behavior remains untested                       |
| Failure absent     | The recorded failure signature did not recur for the specified case              |
| Behavior checked   | Named assertions, outputs, or invariants passed                                  |
| Regressions passed | The identified independent test selection passed, with counts visible            |
| Policy warning     | Deterministic rules found a risky or prohibited change requiring review          |
| Incomplete         | One or more required checks failed, timed out, were skipped, or were unavailable |